

# FIRST-CLASS REFINEMENT TYPES IN SCALA



Matt Bovel, Viktor Kunčak and Martin Odersky

October 3, 2026

# INTRODUCTION

Refinement types are types qualified with logical predicates.

For example,

$$\{x : \text{Int} \mid x > 0\}$$

denotes the type of all integers `x` such that `x > 0`.

Implemented in many languages: [Liquid Haskell](#), [Boolean refinement types in F\\*](#), [Subset types in Dafny](#), etc.

This talk presents:

1. **Implementation** (§2, §4): a prototype implementation in the Scala 3 compiler, as *first-class* Scala types that participate in subtyping, inference, overloading and pattern matching.
2. **Metatheory** (§3): a core calculus proven sound in Rocq by semantic typing, combining refinement types with subtyping, bounded polymorphism, unions, intersections and equi-recursive types, where both programs and predicates may diverge.

PART 1

# IMPLEMENTATION



## SYNTAX (§2.1)

**Long form**, with an explicit binder, mirroring set-builder notation:

```
type NonEmpty[A] = {l: List[A] with  
  l.nonEmpty}
```

- `l` : binder
- `List[A]` : parent type
- `l.nonEmpty` : qualifier (predicate)

**Short form**, reusing a name already in scope:

```
val x: (Int with x % 2 == 0) = 42  
// desugars to:  
val x: {v: Int with v % 2 == 0} = 42
```

```
def zip[A, B](xs: List[A], ys: List[B] with  
  ys.size == xs.size):  
  {l: List[(A, B)] with l.size == xs.size}
```

```
def concat[T](xs: List[T], ys: List[T]):  
  {res: List[T] with res.size == xs.size +  
  ys.size}
```

```
val xs: List[Int] = ...  
val ys: List[Int] = ...  
zip(concat(xs, ys), concat(ys, xs))  
zip(concat(xs, ys), concat(xs, xs)) // error
```

# WHY *FIRST-CLASS*? (§1)

Liquid Haskell is a plugin that runs **after** type checking. The type of `x` is declared twice:

```
{-@ x :: {v:Int | v mod 2 == 0} @-}  
let x = 42 :: Int in ...
```

“It’s sort of like you’re doing two things at once [...] you’re also talking to GHC, but you’re also talking to LiquidHaskell.”

Usability Barriers for Liquid Types (Gamboa et al., PLDI 2025)

One type system, one inference algorithm, one set of error messages:

**Bounded polymorphism**, with no abstract refinements:

```
type Even = {v: Int with v % 2 == 0}  
def maximum[T: Ordering, U <: T]  
  (xs: List[U]): U = xs.reduce(max)  
def example1: Even =  
  maximum(List(2, 4, 6))
```

**Overload resolution:**

```
def min(l: List[Int] with l.isSorted) =  
  l.head // O(1)  
def min(l: List[Int]) = l.min // O(n)  
  
def ex2(l: List[Int] with l.isSorted) =  
  min(l) // calls the first overload
```

## MIXED-PRECISION TYPE INFERENCE (§2.4)

Refinement types are **not** inferred from terms by default; the wider type is:

```
val x = 42 // Int, not {v: Int with v == 42}
```

Why not always infer the precise type?

1. **Backward compatibility:** implicit and overload resolution depend on inferred types; a more precise type changes which instances are found.
2. **Performance:** bigger types, slower comparisons.
3. **Usability:** nested unions of singletons are harder to read than `Int`.

Precision is recovered **on demand**, by two standard mechanisms.

**Equality facts.** T-LET records  $x \sim a$ , “both sides evaluate to the same value”:

$$\frac{\Gamma \vdash a : A \quad \Gamma, x : A, x \sim a \vdash b : B}{\Gamma \vdash \text{let } x:A = a \text{ in } b : \text{avoid}(B, x)}$$

**Selfification.** T-SELF lifts an expression into a type when a refinement is expected:

$$\frac{\Gamma \vdash a : A \quad \text{firstorder}(A)}{\Gamma \vdash a : \{x : A \mid x == a\}}$$

```
val mPlusN = m + n // : Int
// fact mPlusN ~ m + n recorded
Vec.fill(mPlusN, "").zip(v3)
```

## RUN-TIME CHECKS (§2.5)

**Pattern matching** branches on whether the predicate holds at run time:

```
type ID = {s: String with s.matches(idRegex)}  
  
"a2e7-e89b" match  
  case id: ID => ... // holds, id has type ID  
  case _      => ... // does not hold
```

Reuses Scala's existing pattern-matching infrastructure: the type-test lowering phase compiles a match against  $\{x : A \mid p\}$  into a test against  $A$  followed by an evaluation of  $p$ .

**Checked casts.** `.runtimeChecked` (SIP-57), when you expect the check to pass:

```
val id: ID = "a2e7-e89b".runtimeChecked  
  
// desugars to:  
val id: ID =  
  if ("a2e7-e89b".matches(idRegex))  
    "a2e7-e89b".asInstanceOf[ID]  
  else throw new IllegalArgumentException()
```

As with other Scala types, `.asInstanceOf[ID]` skips the check entirely, at the programmer's risk.

## SOLVER (§4.3)

No external SMT dependency: a self-contained solver of  $\approx 600$  lines, light enough to run on every keystroke in an IDE.

To decide  $P_1 \Rightarrow P_2$ : collect the relevant facts (qualifiers, `val` equalities, branch conditions), insert them into an e-graph merged with `true`, and check whether  $P_2$  becomes equivalent to `true`.

**Acyclic e-graph**, as in [Craneflight](#): each class eagerly collapses to one canonical representative instead of keeping all equivalent terms. Merging re-canonicalizes parents, which may cascade. This repair loop is congruence closure.

Domain-specific rewrites during canonicalization:

```
{v: Int with v == x + 3 * y}
  <: {v: Int with v == 2*y + (x + y)}
```

```
{v: Int with v == a && a == b}
  <: {v: Int with v == b}
{v: Int with a == b}
  <: {v: Int with f(a) == f(b)}
```

- **Arithmetic**: flatten sums and products, group like terms, constant-fold ( $2x + 3x \rightarrow 5x$ ). No distribution, to avoid blowup.
- **Boolean**: short-circuiting; a conjunction merged with `true` merges each conjunct with `true`.
- **Beta and constructor-field reduction**:  
`Range(0, 10).from`  $\rightarrow$  0.
- **Transitivity of** `<`.

## EVALUATION (§4.4)

**Compilation overhead** (JMH, single-shot, 21 benchmarks, vs. each system's own unchecked baseline):

| System             | Overhead |
|--------------------|----------|
| First-class (ours) | 0–12%    |
| Schmid and Kunčak  | 20–38%   |
| Stainless          | ≥ 56%    |

*Fansi*, a 1 000-line terminal-color library: three run-time `require` checks replaced by one alias,

```
type ChannelValue =  
  {v: Int with 0 <= v && v < 256}
```

verified statically with no other change. Difference within measurement noise.

**When the feature is unused**, the full Dotty CI passes unchanged: the compiler itself ( $\approx 200\,000$  LoC),  $\approx 10\,000$  tests, a 50-project community build. At most 4% slowdown on the official benchmark suite.

**Expressiveness.** Because refinements are types, we get type-argument inference, bounded polymorphism, pattern matching and overloading by construction. We also compile unmodified Scala, so adoption is incremental.

Conversely, both alternatives have far stronger solvers: SMT-backed and complete for linear integer arithmetic, ADTs and equality. Benchmarks needing linear arithmetic (`sumnat`, `arrfold`) are out of reach for our e-graph.

PART 2

# METATHEORY



## LANGUAGE (§3.1)

Essentially System  $F_{<:>}$  with refinements, dependent functions and pairs, sums, unions, intersections and equi-recursive types:

$$\begin{aligned} A, B ::= & X \mid \mathbf{Unit} \mid \mathbf{True} \mid \mathbf{False} \mid \mathbf{Int32} \mid \top \mid \perp \\ & \mid \Pi x:A. B \mid \forall (X:>L<:U). A \mid \Sigma x:A. B \\ & \mid A + B \mid \{x : A \mid p\} \mid A \vee B \mid A \wedge B \\ & \mid \mu X. A \end{aligned}$$

Types and terms are **mutually inductive**: a refinement embeds a term.

$$\begin{aligned} a, b, f, p ::= & c \mid x \mid \lambda x:A. b \mid f \ a \mid f \ [A] \\ & \mid \Lambda (X:>L<:U). b \mid \mathbf{let} \ x:A = a \ \mathbf{in} \ b \\ & \mid (a_1, a_2) \mid \mathbf{inl}[A] \ a \mid \mathbf{inr}[A] \ a \mid \mathbf{match} \dots \\ & \mid a \ \mathit{op} \ b \mid \mathbf{if} \ a \ \mathbf{then} \ b_1 \ \mathbf{else} \ b_2 \mid \mathbf{loop}(a) \ x. b \end{aligned}$$

$\mathbf{loop}(a) \ x. b$  is a limited recursion that suffices for tail recursion: the body returns  $\mathbf{inl}$  to continue,  $\mathbf{inr}$  to exit.

$$\Gamma ::= \emptyset \mid \Gamma, x : A \mid \Gamma, X :> L <: U \mid \Gamma, a_1 \sim a_2$$

Three kinds of entries. Note the **equality facts**  $a_1 \sim a_2$ .

To our knowledge, the first mechanized soundness proof combining refinements with  $\vee/\wedge$ , bounded polymorphism (both bounds), and positive equi-recursive types.

## DEFINITIONAL INTERPRETER (§3.2)

Evaluation is a **fuel-bounded definitional interpreter** in Rocq, following Amin and Rompf:

```
Fixpoint eval (fuel: nat) (env: list Value)
  (t: Term) : option (option Value) :=
  match fuel with
  | 0 => None (* timeout *)
  | S fuel' =>
    match t with
    | tabs _ b =>
      Some (Some (vabs env b))
    | tapp f a =>
      match eval fuel' env f with
      | Some (Some (vabs envf b)) =>
        ... eval fuel' (va :: envf) b
      | None => None (* timeout *)
      | _ => Some None (* stuck *)
    end
  end
...

```

Three outcomes, not two:

- `None` : timeout (fuel exhausted),
- `Some None` : **stuck**,
- `Some (Some v)` : **success**.

**Fuel bounds depth, not steps.** For  $f\ a$ , the function, argument and body all get the *same* decremented fuel.

Simpler, since it is a structural termination measure, but it gives no global step count to correlate with recursive-type unfoldings. We side-step that by making the value interpretation step-index-free.

## INTERPRETATION (§3.3)

A semantic type is a predicate on values,  $\text{Value} \rightarrow \text{Prop}$ .  $\mathcal{V}[[A]]_\delta^\rho$  maps a syntactic type to one, given a type assignment  $\delta$  and a value assignment  $\rho$ :

$$\begin{aligned} \mathcal{V}[[\Pi x:A. B]]_\delta^\rho(v) &\triangleq \exists \rho_f, b. v = \langle \rho_f, \lambda x. b \rangle \wedge \\ &\quad \forall v_a. \mathcal{V}[[A]]_\delta^\rho(v_a) \implies \\ &\quad \mathcal{E}[[B]]_\delta^{\rho_f[x \mapsto v_a]}(b) \end{aligned}$$

$$\mathcal{V}[\{x : A \mid p\}]_\delta^\rho(v) \triangleq \mathcal{V}[[A]]_\delta^\rho(v) \wedge \mathcal{E}[[\text{True}]]_\delta^{\rho[x \mapsto v]}(p)$$

$$\mathcal{V}[[\mu X. A]]_\delta^\rho(v) \triangleq \forall n. F^n(v)$$

$$F^0(v) = \top, \quad F^{n+1}(v) = \mathcal{V}[[A]]_{\delta[X \mapsto F^n]}^\rho(v)$$

The term interpretation lifts it to terms. This is where **partial correctness** lives:

$$\begin{aligned} \mathcal{E}[[A]]_\delta^\rho(a) &\triangleq \forall n, r. \text{eval } n \ \rho \ a = \text{Some } r \implies \\ &\quad \exists v. r = \text{Some } v \wedge \mathcal{V}[[A]]_\delta^\rho(v) \end{aligned}$$

“If evaluation terminates, it produces a value, not a stuck state, and that value is in  $\mathcal{V}[[A]]$ .” Vacuously true for diverging terms.

Well-formedness ties the assignments to the context:

$$\begin{aligned} \text{wf}(\delta, \Gamma, \rho) &\triangleq \forall (x : A) \in \Gamma. \mathcal{V}[[A]](\rho(x)) \\ &\quad \wedge \forall (X :> L <: U) \in \Gamma. \\ &\quad \quad \mathcal{V}[[L]] \subseteq \delta(X) \subseteq \mathcal{V}[[U]] \\ &\quad \wedge \forall (a_1 \sim a_2) \in \Gamma. \exists v, n_1, n_2. \\ &\quad \quad \text{eval } n_1 \ \rho \ a_1 = \text{eval } n_2 \ \rho \ a_2 \\ &\quad \quad = \text{Some } (\text{Some } v) \end{aligned}$$

## TYPING (§3.4)

Semantic typing is **defined**, not derived: it quantifies over every well-formed environment:

$$\Gamma \models a : A \triangleq \forall \delta, \rho. \text{wf}(\delta, \Gamma, \rho) \implies \mathcal{E}[[A]]_{\delta}^{\rho}(a)$$

So the usual typing rules are not definitions but **lemmas**: each of the 20 rules is proven individually, and any derivation built from them inherits safety. This is the fundamental theorem of the logical relation.

A syntactic judgment  $\Gamma \vdash a : A$  mirrors the rules, with:

**Theorem (Adequacy).** If  $\Gamma \vdash a : A$  then  $\Gamma \models a : A$ .

$$\frac{\Gamma \models a : A \quad \Gamma, x : A, x \sim a \models b : B}{\Gamma \models \text{let } x:A = a \text{ in } b : \text{avoid}(B, x)} \text{ (T-Let)}$$

$$\frac{\Gamma \models a : A \quad \text{firstorder}(A)}{\Gamma \models a : \{x : A \mid x == a\}} \text{ (T-Self)}$$

$$\frac{\Gamma \models a : A \quad \Gamma, x : A \models b : A + B}{\Gamma \models \text{loop}(a) \ x. b : B} \text{ (T-Loop)}$$

## SUBTYPING (§3.5)

Subtyping is semantic inclusion, again quantified over all well-formed environments:

$$\Gamma \models A <: B \triangleq \forall \delta, \rho. \text{wf}(\delta, \Gamma, \rho) \implies \mathcal{V}[[A]]_\delta^\rho \subseteq \mathcal{V}[[B]]_\delta^\rho$$

Every value satisfying  $A$  also satisfies  $B$ . The 20 subtyping rules are, again, lemmas.

$$\Gamma \models \{x : A \mid p\} <: A \text{ (S-RefineBase)}$$

$$\frac{\Gamma \models A <: B \quad \Gamma, x : A \models p_1 \Rightarrow p_2}{\Gamma \models \{x : A \mid p_1\} <: \{x : B \mid p_2\}} \text{ (S-Refine)}$$

Recursive types are **equi**-recursive: no fold/unfold terms, just subtyping.

$$\frac{\text{spos}(X, A)}{\Gamma \models \mu X. A <: A[X \mapsto \mu X. A]} \text{ (S-Mu-Unfold)}$$

$$\frac{\text{spos}(X, A)}{\Gamma \models A[X \mapsto \mu X. A] <: \mu X. A} \text{ (S-Mu-Fold)}$$

Together they give  $\mu X. A \cong A[X \mapsto \mu X. A]$ , proven by showing the two interpretations coincide. That is the ingredient that replaces step indexing.

**Strict positivity** buys that:  $X$  never occurs left of an arrow or in a  $\forall$  bound. Two sharper edges:

- under a **union**,  $X$  must be entirely absent;
- under a **nested**  $\mu Y. B$ , we need  $\text{spos}(Y, B)$  and

## SEMANTIC IMPLICATION AND TERMINATION (§2.3, §3.3)

Predicates are ordinary pure terms, so they **may diverge**.  $\{x : A \mid p\}$  holds when  $p$  evaluates to `true` *whenever it terminates*, exactly like a run-time assertion, which only fails if the check returns `false`.

Termination is confined to **one** judgment, used only by S-REFINE:

$$\Gamma \models p_1 \Rightarrow p_2 \triangleq \forall \delta, \rho. \text{wf}(\delta, \Gamma, \rho) \implies \mathcal{E}[\text{True}](p_1) \implies \mathcal{E}[\text{True}](p_2)$$

No typing rule mentions termination, so soundness needs **no termination assumption**, unlike System FR (total correctness), Liquid Haskell (reflected functions), F\* and Dafny (specification-level computation).

Some entailments need nothing:

**Lemma (And-True).**  $\Gamma \models (t \ \&\& \ \text{true}) \Rightarrow t$ .

Others genuinely do:

**Lemma (And-False).**  $t$  terminates  $\implies \Gamma \models (t \ \&\& \ \text{false}) \Rightarrow \text{false}$ .

Alarming? No: a diverging predicate is indistinguishable from `true`, not from `false`, and `true` proves nothing:

**Lemma.**  $\Gamma \models \{x : A \mid \text{diverge}\} <:\> \{x : A \mid \text{true}\} <:\> A$ , and no *terminating* term has type  $\{x : A \mid \text{false}\}$ .

# FUTURE WORK

## Implementation

- **Flow-sensitive typing.** Works for patterns today;

`if` conditions should narrow too:

```
if x > 0 then x: {v: Int with v > 0}
```

- **External checks** for what our lightweight solver cannot do. For example, it cannot yet prove

```
{v: Int with v > 2} <: {v: Int with v > 0}:
```

```
x.externallyChecked: {v: Int with v > 0}
```

- **Term-parameterized types**, to modularize predicates:

```
type Range(from: Int, to: Int) = {v: Int with  
v >= from && v < to}
```

## Theory

- **Enforcing purity.** Today a function called in a predicate is *assumed* pure; an impure one is a silent logical error. Capture checking tracks exactly this, and separation checking would make invalidating facts on writes sound.
- **A syntactic proof system for entailment.** We leave implication purely semantic; the solver's normalization rules are not yet verified against it.
- **A termination checker**, needed only for the termination-sensitive entailment rules, never systematically for every function in a predicate.
- **Classes and objects**, absent from the core

# SUMMARY

Refinement types can be **first-class types** in a mainstream language.

- **Design:** refinements are ordinary Scala types, so subtyping, inference, bounded polymorphism, overloading and pattern matching work by construction.
- **Partial correctness:** predicates may diverge; a diverging predicate is vacuous, and soundness needs no termination assumption.
- **Metatheory:** the first mechanized proof combining refinements with unions, intersections, bounded polymorphism with both bounds, and positive equi-recursive types.

**Try it.** Published on Maven, usable from any SBT project:

```
scalaOrganization := "ch.epfl.lara"  
scalaVersion := "3.10.0-RC1-bin-...-NIGHTLY"  
scalacOptions += Seq(  
  "-language:experimental.qualifiedTypes")  
  
type Even = {v: Int with v % 2 == 0}  
val x: Even = 42
```

**Artifact:** Rocq mechanization, the Dotty extension, and the JMH benchmark suites.

<https://github.com/mbovel/first-class-refinement-types-in-scala>

Thank you. Questions welcome.